

IDX Exchange

SDE Internship — Challenge Guide

12-Week Project | Build a Property Search Application

Last revised: 4/28/2026

How to Use This Guide

This is the challenge version of the IDX internship project. Unlike the training guide, this document does NOT hand you working code. Instead, each week gives you:

- **Requirements** — what the feature must do
- **Acceptance Criteria** — how you and your supervisor will verify it works
- **Hints** — nudges when you are genuinely stuck (try without them first)
- **Debug Challenges** — intentional bugs you must find and fix

You are expected to write all the code yourself. Use the official documentation for Express, React, MySQL2, and React Router. If you are completely stuck for more than 30 minutes, ask your supervisor — not for the answer, but for a direction.

⚠ The training guide exists as a reference. Looking at it defeats the purpose. Use it only as a last resort after exhausting documentation, Stack Overflow, and asking your supervisor.

Project Overview

What You Are Building:

A Zillow/Redfin-style property search experience backed by real MLS data. The finished application must include:

- A searchable, filterable property listings page with pagination
- A property detail page with photos, map, and open house schedule
- A Node/Express REST API connecting React to MySQL
- A local MySQL database populated from two provided SQL files

Technical Stack:

- Frontend: React (Create React App)
- Backend: Node.js + Express
- Database: MySQL 8 running in Docker
- Testing: Jest + React Testing Library + Supertest

Data Flow:

Brain: calls Mysql query when
clicked something in react and
returns to react
 React (port 3000) → Express API (port 5000) → MySQL (port 3306)

⚠ React must never connect directly to MySQL. All data goes through the Express API.

The Two Database Tables:

You are provided two SQL files. Study them before writing any code:

- **rets_property** — property listings. Key columns: L_ListingID, L_Address, L_City, L_State, L_Zip, L_SystemPrice (price), L_Keyword2 (beds), LM_Dec_3 (baths), LM_Int2_3 (sqft), L_Photos (JSON array of URLs), LMD_MP_Latitude, LMD_MP_Longitude, L_Remarks, YearBuilt, LotSizeAcres
- **rets_openhouse** — open house events. Key columns: L_ListingID (foreign key), OpenHouseDate, OH_StartTime, OH_EndTime, all_data (JSON blob containing OpenHouseRemarks and other fields)

⚠ The column names in these tables use an older RETS naming convention and do NOT match standard MLS field names. Read the SQL files carefully — your queries must use the actual column names.

Prerequisites — Install Before Week 1:

- Node.js (LTS version) and npm
 - Docker Desktop
 - Git
 - VS Code (recommended)
-

Week 1: Environment Setup & Database Import

Goal: MySQL running in Docker with both tables populated and queryable.

Download FileZilla Client from <https://filezilla-project.org>

Establish FTP connection using Host: 199.250.207.194, Username: rawfiles@idxexchange.com, Password – getthefiles123! Port: 21 (Type in the password instead of copying and pasting)

Download the `rets_property.sql` and `rets_openhouse.sql` files from the 'sql' folder on your computer.

Requirements

1. Install Docker Desktop and verify it is running
2. Start a MySQL 8 container named `idx-mysql-local` on port 3306 with a database called `rets`
3. Import both `rets_property.sql` and `rets_openhouse.sql` into the `rets` database
4. Verify both tables exist and contain data

Acceptance Criteria

- `docker ps` shows `idx-mysql-local` running
- `SHOW TABLES;` returns `rets_property` and `rets_openhouse`
- `SELECT COUNT(*) FROM rets_property;` returns a non-zero number
- `SELECT COUNT(*) FROM rets_openhouse;` returns a non-zero number
- You can run a `SELECT` on both tables and describe their columns
- You can explain what Docker containers are and why we use one for MySQL

Hints

 **Hint:** The docker run command needs flags for the root password, database name, port mapping, and background mode. Check the Docker Hub MySQL page for the exact environment variable names.

 **Hint:** To import SQL files into a running container, use `docker exec` with the `-i` flag to pipe the file contents into the `mysql` command.

 **Hint:** Use `DESCRIBE rets_property;` to see all column names before you write any backend code. This will save you a lot of pain later.

Week 1 Checkpoint

Before moving on, demo the following to your supervisor:

- ☐ MySQL container starts automatically after docker start `idx-mysql-local`
- ☐ Both tables are populated and you can query them from the MySQL shell

☐ You understand what each column in `rets_property` represents

Week 2: Backend Foundation + REST API Basics

Goal: A running Node/Express server with a working health check endpoint.

Requirements

5. Initialize a Node.js project in a backend/ folder
6. Install express, mysql2, dotenv, and cors. Install nodemon as a dev dependency
7. Create a .env file with your database credentials (never commit this file)
8. Create a MySQL connection pool module
9. Create a GET /api/health endpoint that queries the database and returns connection status
10. Server must auto-restart on file changes during development

Acceptance Criteria

- npm run dev starts the server without errors
- GET /api/health returns { status: "ok", database: "connected" } when MySQL is running
- GET /api/health returns a 500 error (not a crash) when MySQL is unreachable
- .env is listed in .gitignore
- You can explain: What is a connection pool? Why not create a new connection per request?
- You can explain: What is the difference between GET, POST, PUT, and DELETE?
- You can explain: When should an API return 400 vs 404 vs 500?

Hints

💡 **Hint:** mysql2/promise gives you async/await support. Create the pool with mysql.createPool() and pass it your .env values via process.env.

💡 **Hint:** dotenv must be configured before you access process.env. Call require("dotenv").config() early in your entry file.

💡 **Hint:** For the health check, a simple SELECT 1 query is enough to verify the database connection.

Week 2 Checkpoint

- ☐ Server runs on port 5000
- ☐ /api/health returns correct response for both connected and disconnected database
- ☐ You can explain REST principles and HTTP status codes

Week 3: Property Search Endpoint with Filters & Indexing

Goal: A paginated, filterable GET /api/properties endpoint backed by proper database indexes.

Requirements

11. Create a properties route file and mount it at /api/properties
12. Implement GET /api/properties with pagination (limit and offset query params)
13. Add filter support for: city, zipcode, minPrice, maxPrice, beds, baths
14. Validate all query parameters — return 400 with a descriptive message for invalid inputs
15. Create database indexes on the columns you filter by
16. Measure query performance before and after adding indexes using EXPLAIN

API Contract

Your endpoint must accept these query parameters:

```
GET
/api/properties?city=Portland&minPrice=300000&beds=3&limit=20&offset=0
```

And return this shape:

```
{ "total": 87, "limit": 20, "offset": 0, "results": [...] }
```

Acceptance Criteria

- GET /api/properties returns 20 properties by default with a total count
- Pagination works: ?limit=10&offset=20 returns properties 21-30
- Filtering works: ?city=Portland returns only Portland properties
- Multiple filters combine: ?city=Portland&minPrice=300000&beds=3
- Invalid inputs return 400: ?minPrice=abc, ?limit=0, ?limit=200
- All filter values use parameterized queries (no string concatenation)
- SHOW INDEXES FROM rets_property shows your new indexes
- EXPLAIN shows your indexes are being used (key column is not NULL)
- You can explain: Why is SQL injection dangerous? How do parameterized queries prevent it?

Debug Challenge

 **Debug Challenge:** The filter endpoint has a subtle bug: when both minPrice and beds filters are applied together, the result count is wrong. Write a test case

that exposes this bug, then find and fix it. Hint: look carefully at how the values array is built for parameterized queries.

Hints

💡 **Hint:** Build the WHERE clause dynamically by pushing conditions and values into arrays, then joining conditions with AND.

💡 **Hint:** The city column has inconsistent casing in the data. Use LOWER(TRIM()) on both sides of the comparison to handle this.

💡 **Hint:** Remember: the column names in `rets_property` are NOT standard MLS names. Check what they actually are before writing your WHERE clauses.

💡 **Hint:** For composite indexes on frequently combined filters (e.g. city + price), a single CREATE INDEX with multiple columns can be more efficient than two separate indexes.

Week 3 Checkpoint

- ☐ All filter combinations work correctly
 - ☐ Invalid inputs are rejected with helpful error messages
 - ☐ Indexes exist and EXPLAIN confirms they are being used
 - ☐ You can walk through the query-building logic and explain why each part exists
-

Week 4: Property Detail & Open House Endpoints

Goal: Two new endpoints — property by ID and open houses by property ID.

Requirements

17. Implement GET /api/properties/:id — returns a single property or 404
18. Implement GET /api/properties/:id/openhouses — returns open house events for a property
19. Validate the listing ID parameter on both endpoints
20. Add request logging middleware (method, URL, timestamp, response time)

Acceptance Criteria

- GET /api/properties/:id returns the full property object
- GET /api/properties/:id returns 404 with a helpful message for unknown IDs
- GET /api/properties/:id/openhouses returns an array (empty array is OK, not an error)
- Open houses are ordered by date and start time
- Malformed or oversized IDs return 400
- Every request prints a log line with method, URL, status code, and duration in ms
- Route order is correct — /openhouses must be registered before /:id
- You can explain why route order matters in Express

Debug Challenge

 **Debug Challenge:** The open houses endpoint works for most properties but crashes with an unhandled promise rejection for one specific listing ID. Add proper error handling, then figure out what is special about that listing ID that causes the crash.

Hints

💡 **Hint:** Express matches routes in the order they are defined. If /:id is registered first, a request to /abc123/openhouses will match it with id = "abc123/openhouses" — or fail entirely.

💡 **Hint:** For the openhouses endpoint, first verify the property exists, then query `rets_openhouse` by `L_ListingID`.

💡 **Hint:** The `all_data` column in `rets_openhouse` is a JSON blob. You do not need to parse it in the backend — return it as-is and let the frontend handle it, or use MySQL's `JSON_EXTRACT()` function if you want specific fields.

Week 4 Checkpoint

- ☐ Both endpoints return correct data
 - ☐ Error cases are handled (404, 400, 500)
 - ☐ Request logs appear in the terminal with timing information
-

Week 5: React Setup & Listings Page

Goal: A React frontend that fetches and displays a grid of property cards.

Requirements

21. Create a React app in a frontend/ folder
22. Configure a proxy so API calls to /api/* go to your Express server
23. Create an API client module with functions for fetching properties
24. Build a ListingsPage component with a grid of PropertyCard components
25. Each PropertyCard must display: first photo (parsed from L_Photos JSON array), price, address, city/state, beds, baths, and sqft
26. Handle loading and error states

Acceptance Criteria

- React app runs on port 3000 without errors
- Property grid shows cards with real data from the database
- L_Photos is correctly parsed as a JSON array — the first photo URL is displayed
- Loading state shows while data is fetching
- Error message shows if the backend is unreachable
- Property count (e.g. "Showing 20 of 487 properties") is displayed
- Cards have a hover effect
- API client handles HTTP errors and throws meaningful error messages

Debug Challenge

 **Debug Challenge:** After wiring up the PropertyCard, images are broken for some properties but not others. Investigate: is the L_Photos data always a valid JSON array? What does it contain for properties with no photos? Write defensive code that handles all cases without crashing.

Hints

 **Hint:** L_Photos is stored as a JSON string in the database. Use JSON.parse() on it, but wrap it in try/catch — not all rows have valid JSON.

 **Hint:** The proxy setting in package.json only works during local development. Set "proxy": "http://localhost:5000" at the top level of package.json, then restart the dev server.

 **Hint:** Create an api/ folder with a client.js module. Export individual functions like fetchProperties(params) and fetchPropertyDetail(id). This keeps your components clean.

Week 5 Checkpoint

- ☐ Property grid renders with real photos, prices, and addresses
 - ☐ Photo parsing handles missing, null, and malformed L_Photos values gracefully
 - ☐ API errors are caught and displayed to the user
-

Week 6: Filters UI + Introduction to Testing

Goal: A working filter form and your first unit tests.

Requirements

27. Build a PropertyFilters component with inputs for: city, ZIP code, min price, max price, beds (dropdown), baths (dropdown)
28. Integrate filters into the ListingsPage — searching updates the property list
29. A "Clear Filters" button resets the form and reloads all properties
30. Write at least 3 unit tests for your API client module
31. Write at least 3 unit tests for the PropertyFilters component

Acceptance Criteria

- Filter form displays all six inputs
- Submitting the form fetches new results matching the filters
- Multiple filters can be combined
- Empty filter values are not sent to the API
- Clear button resets the form and results
- No properties found state shows a helpful message
- npm test passes all tests
- You can explain: What is a unit test? What is mocking? Why do we mock fetch()?

Debug Challenge

 **Debug Challenge:** There is a subtle UX bug: when the user types a city, clicks Search, then clicks Clear, then types a new city and clicks Search — the results from the first search briefly flash before being replaced. Identify why this happens and fix it. Hint: think about async timing and React state updates.

Hints

💡 **Hint:** Control all filter inputs with React state. Use a single object for all filter values and update it with the spread operator.

💡 **Hint:** Remove empty string values from the filter object before passing it to the API — otherwise the backend receives empty parameters that may affect query results.

💡 **Hint:** For tests, use `jest.fn()` to mock `global.fetch` and control what it returns. Test both the success path and the error path.

Week 6 Checkpoint

- ☐ All filter combinations return correct results
 - ☐ Clear filters works correctly in all sequences
 - ☐ All tests pass with npm test
-

Week 7: Pagination UI & Component Testing

Goal: Working pagination controls and comprehensive component tests.

Requirements

- 32. Add pagination state to ListingsPage (currentPage, itemsPerPage)
- 33. Build a Pagination component that displays page numbers, previous/next buttons, and ellipsis for large page counts
- 34. Changing pages must scroll to top and preserve active filters
- 35. Changing filters must reset to page 1
- 36. Write tests covering all Pagination component behaviors

Acceptance Criteria

- Pagination controls appear below the property grid
- Previous is disabled on page 1; Next is disabled on the last page
- Clicking a page number navigates to that page
- Page numbers use ellipsis correctly for large counts (e.g. 1 ... 4 5 6 ... 24)
- Results summary shows "Showing X-Y of Z properties"
- Applying new filters resets to page 1
- Pagination is hidden when there is only one page
- All component tests pass

Debug Challenge

 **Debug Challenge:** The pagination component renders "1 ... 2 3 4 ... 1" (the last page number appears twice) when on certain pages near the end. Write a test case that reproduces this, then fix the page number generation logic.

Hints

- 💡 **Hint:** The page number generation logic is the trickiest part. Handle three cases: total pages fit without ellipsis, current page is near the start, current page is near the end, and current page is in the middle.
- 💡 **Hint:** Use `window.scrollTo(0, 0)` to scroll to the top when the page changes.
- 💡 **Hint:** React Testing Library's `fireEvent.click()` is what you need to simulate button presses in tests.

Week 7 Checkpoint

- ☐ All pagination edge cases work correctly

- ☐ Tests cover: first page, last page, middle page, page number click, ellipsis rendering
 - ☐ Filters and pagination interact correctly
-

Week 8: Property Detail Page End-to-End

Goal: A full property detail page with routing, photo gallery, map, and open houses.

Requirements

37. Install React Router and set up routes: / for ListingsPage, /property/:id for PropertyDetailPage
38. Make property cards clickable — clicking navigates to the detail page
39. Build PropertyDetailPage displaying: price, address, stats (beds/baths/sqft/year built), description, property details, and open houses
40. Build a PropertyImageCarousel component for listing cards (multiple photos, prev/next arrows, counter)
41. Build a PropertyImageGallery component for the detail page (main image, thumbnail strip, lightbox on click)
42. Both photo components must parse L_Photos from its JSON array format
43. Build a PropertyMap component using the Google Maps Embed API (iframe-based, no npm package)
44. Display the map on the detail page using LMD_MP_Latitude and LMD_MP_Longitude
45. Display open houses with date, start/end time, and remarks (remarks are inside the all_data JSON field)

Google Maps Setup

To use the Google Maps Embed API you need an API key:

46. Go to <https://console.cloud.google.com> and sign in
47. Create a new project
48. Enable the Maps Embed API under APIs & Services > Library
49. Create an API key under APIs & Services > Credentials
50. Add the key to frontend/.env as
REACT_APP_GOOGLE_MAPS_API_KEY=your_key
51. Restrict the key to localhost:3000 and the Maps Embed API only

💡 **Hint:** React env variables must start with REACT_APP_ to be accessible in the browser. Restart the dev server after editing .env.

Acceptance Criteria

- Clicking a card navigates to /property/[listing-id]
- Back button returns to the listings page
- Detail page shows all property fields listed above

- PropertyImageCarousel: arrow buttons cycle through photos, counter shows X / Y
- PropertyImageCarousel: arrows do not navigate to the detail page (stopPropagation)
- PropertyImageGallery: thumbnail strip scrolls, clicking a thumbnail updates main image
- PropertyImageGallery: clicking main image opens a full-screen lightbox
- Lightbox closes on click-outside or Escape key; left/right arrows navigate photos
- PropertyMap renders an iframe with the correct property location
- Map only renders when both lat and lng are present
- Get Directions link opens Google Maps in a new tab
- Open houses show date, formatted times, and remarks if available
- If no open houses, shows "No open houses scheduled"
- Visiting /property/invalid-id shows an error, not a crash

Debug Challenges

 **Debug Challenge:** After wiring up the open house component, remarks never appear even for open houses that have them. The all_data column stores a JSON blob — figure out how to extract OpenHouseRemarks from it in the React component without changing the backend.

 **Debug Challenge:** The lightbox does not close when the user presses Escape. The keydown handler is attached but never fires. Research why keyboard events on a div require a specific attribute to work, and fix it.

Hints

 **Hint:** For the carousel and gallery, parse L_Photos with JSON.parse() inside a try/catch. Fall back to an empty array on failure.

 **Hint:** The Google Maps Embed URL format is:
<https://www.google.com/maps/embed/v1/place?key=KEY&q=LAT,LNG&zoom=15>

 **Hint:** For the open house remarks, JSON.parse() the all_data string, then access the OpenHouseRemarks key. Do this in the component, not the API.

 **Hint:** For keyboard events on a div, check what attributes are required for the element to be focusable and receive keyboard events.

Week 8 Checkpoint

- ☐ Full navigation flow works: listings → detail → back
- ☐ Photo carousel and gallery work with multi-photo and single-photo properties

- ☐ Lightbox opens, navigates, and closes correctly including keyboard
 - ☐ Map renders with correct location and directions link works
 - ☐ Open house remarks display correctly
-

Week 9: Advanced Feature (Required) + Performance Optimization

Goal: Implement one required advanced feature and optimize application performance.

Part A — Advanced Feature (Pick One, Complete It Fully)

You must pick ONE of the following options and implement it completely. "Partially implemented" does not meet the checkpoint requirement.

Option 1: Sorting (Estimated: 4-5 hours)

Allow users to sort properties by price, date listed, square footage, or beds.

Requirements:

- Backend: Accept sortBy and sortOrder query parameters
- Backend: Validate sortBy against a whitelist of actual column names from rets_property (not RESO names)
- Frontend: Add sort controls to the listings page
- Frontend: Sort state must be preserved when changing pages, but reset when filters change

⚠ The sort field whitelist must use the actual SQL column names (e.g. L_SystemPrice, not ListPrice). Using wrong names will silently return unsorted results.

Acceptance Criteria:

- Sorting by price low-to-high and high-to-low works correctly
- Sorting by date listed works correctly
- Invalid sortBy values are rejected with a 400 error
- Sort persists across page changes
- Sort resets when new filters are applied

Option 2: Favorites (Estimated: 5-6 hours)

Let users save and view favorite properties using a custom React hook.

Requirements:

- Create a useFavorites custom hook that persists favorites in localStorage
- Add a heart button to each PropertyCard
- Heart click must not trigger card navigation (stopPropagation)
- Add a "Favorites" view showing only saved properties

Acceptance Criteria:

- Favoriting a property persists across page refreshes
- Unfavoriting removes the property immediately from the Favorites view
- The heart icon shows filled/empty state correctly
- Favorites count is shown somewhere in the UI
- A custom hook is used (not inline localStorage calls in the component)

Option 3: Sorting + Favorites (Estimated: 8-10 hours)

Implement both Option 1 and Option 2 for maximum challenge.

Option 4: Open House Calendar (Estimated: 8-10 hours)

Display all upcoming open houses in a calendar view.

Requirements:

- Add a new backend endpoint: GET /api/openhouses?startDate=&endDate=
- Use react-big-calendar or build your own week/month grid
- Clicking an event navigates to the property detail page
- Calendar must handle the all_data JSON field correctly for event details

Acceptance Criteria:

- Calendar displays open houses on correct dates
- Date range filtering works on the backend
- Clicking an event navigates to the correct property
- Empty date ranges return an empty array (not an error)

Option 5: Natural Language Property Search (Estimated: 10-14 hours)

Build a second search page where users type plain English queries instead of filling out filter forms. A query like "3 bedroom house in Portland under \$500k with a pool" should return matching properties without the user touching a single dropdown.

This feature uses the Anthropic Claude API to parse natural language into structured filter parameters, which are then passed to your existing /api/properties endpoint.

⚠ This option requires an Anthropic API key. Sign up at <https://console.anthropic.com> — new accounts receive free credits sufficient for this project. Never commit your API key to Git.

How It Works:

- User types: "3 bed 2 bath house in Beverly Hills under \$800k built after 2000"
- Your backend sends that text to the Claude API with a system prompt that instructs it to extract filter fields

- Claude returns structured JSON: { city: "Beverly Hills", minBeds: 3, minBaths: 2, maxPrice: 800000, minYearBuilt: 2000 }
- Your backend validates the extracted fields and passes them to the existing properties query
- Results are returned to the frontend just like a regular search

Requirements:

- Add ANTHROPIC_API_KEY to your backend .env file
- Create a POST /api/search/natural endpoint that accepts { query: string }
- The endpoint calls the Claude API (claude-sonnet-4-20250514) to extract filter parameters from the query
- Extracted parameters must be validated before being passed to the database — Claude can return unexpected values
- Create a NaturalSearchPage React component with a single text input and search button
- Results display using the same PropertyCard components as the main listings page
- Show the interpreted filters below the search box so users can see what was extracted (e.g. "Searching: city=Beverly Hills, max price=\$800,000, beds=3+")
- Handle the case where Claude cannot extract any valid filters — show a helpful message, not an empty grid
- Add the new page as a route: /search/natural
- Add a navigation link to it from the main listings page

Backend Implementation Guide:

Your POST /api/search/natural endpoint should:

- Accept a JSON body with a query string field
- Call the Anthropic API with a carefully written system prompt that instructs Claude to extract property filter fields and return ONLY valid JSON — no prose, no markdown fences
- Parse Claude's JSON response and map extracted fields to your actual SQL column names
- Validate each extracted value (price must be numeric, beds must be a positive integer, city must be a non-empty string, etc.)
- Pass the validated fields to the same query-building logic used in GET /api/properties
- Return both the results AND the interpreted filters so the frontend can display them

Your system prompt to Claude should specify the exact JSON schema it must return. The schema should map to fields your query builder already understands:

```
{ city, zipcode, minPrice, maxPrice, beds, baths, minYearBuilt,
  maxYearBuilt }
```

⚠️ Claude may return fields with null, undefined, or unexpected types. Your validation layer must handle all of these before touching the database. Never pass unvalidated AI output directly to a SQL query.

Install the Anthropic SDK:

```
npm install @anthropic-ai/sdk
```

Basic structure for calling the API from your Express route:

```
const Anthropic = require('@anthropic-ai/sdk');
const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY
});

const message = await client.messages.create({
  model: 'claude-sonnet-4-20250514',
  max_tokens: 1024,
  system: yourSystemPrompt,
  messages: [{ role: 'user', content: userQuery }]
});

const extracted = JSON.parse(message.content[0].text);
```

💡 **Hint:** Writing a good system prompt is the core challenge here. It needs to tell Claude exactly what fields to extract, what format to return them in, and what to do when the query is ambiguous or missing information. Experiment with varied inputs like "cheap houses", "something big with a garage", and "4 beds in 90210" to stress-test your prompt.

Acceptance Criteria:

- POST /api/search/natural accepts a natural language query and returns property results
- "3 bedroom house in [city] under \$[price]" correctly extracts city, beds, and maxPrice
- Extracted filters are displayed to the user below the search box
- Vague or unextractable queries return a helpful message, not a 500 error
- Unvalidated AI output never reaches the SQL query
- ANTHROPIC_API_KEY is in .env and in .gitignore — never hardcoded or committed
- NaturalSearchPage is accessible at /search/natural with a nav link from the listings page
- Results use the same PropertyCard components as the main listings page
- Anthropic API errors (rate limit, invalid key, timeout) are caught and return a 503 with a user-friendly message
- You can explain: What is a system prompt? Why must AI output be validated before hitting the database?

Debug Challenges:

 **Debug Challenge:** Claude correctly extracts the city name but results come back empty even though matching properties exist. Trace the full path from Claude's output to the SQL WHERE clause — the casing or whitespace of the extracted city does not match the database. Fix the normalization.

 **Debug Challenge:** For the query "cheap houses with lots of space", Claude returns null for all numeric fields. Your validation passes them through anyway and the SQL query breaks. Write a test that catches this and fix the validation to strip out null and undefined values before query-building.

Extension Ideas (if you finish early):

- Add a loading indicator while Claude processes the query
- Store recent natural language searches in localStorage and show them as suggestions
- Support follow-up refinements: "show me cheaper ones" or "only with a pool" — maintain conversation context across queries using the messages array
- Show a confidence breakdown of which fields Claude extracted with certainty vs. inferred

Part B — Performance Optimization

Complete all of the following regardless of which advanced feature you chose:

52. Run EXPLAIN on your most complex filter query and interpret the output — document what each column means
53. Add composite indexes for the most common filter combinations and measure the improvement
54. Enhance the request logging middleware to include response time in milliseconds
55. Add a React Error Boundary that catches render errors and shows a recovery UI
56. Identify and fix any console warnings in the React app

Week 9 Checkpoint

- ☐ Chosen advanced feature is fully functional — all acceptance criteria met
 - ☐ EXPLAIN output documented and indexes added based on findings
 - ☐ Request logs show timing information
 - ☐ Error boundary implemented and tested
 - ☐ No console warnings
-

Week 10: Git Workflow & Code Organization

Goal: Professional Git history and a well-organized, maintainable codebase.

Requirements

- 57. Initialize a Git repository with a meaningful initial commit
- 58. Create a develop branch; all feature work goes on feature branches off develop
- 59. Make at least 5 meaningful commits with conventional commit message format
- 60. Create a pull_request_template.md
- 61. Extract PropertyCard into its own component file with PropTypes
- 62. Organize frontend/src into: api/, components/, pages/, hooks/, utils/
- 63. Remove all console.log statements, commented-out code, and unused imports
- 64. Linter must pass with no errors

Conventional Commit Format

Every commit message must follow this format:

```
type(scope): short description
```

Valid types: feat, fix, refactor, test, docs, chore

Examples:

```
feat(filters): add beds and baths filter to listings page
fix(pagination): reset to page 1 when filters change
refactor(api): extract validation into shared helper function
test(pagination): add edge case tests for ellipsis generation
```

Acceptance Criteria

- git log shows at least 5 commits with conventional format
- At least 3 feature branches created and merged into develop
- No commits directly to main
- Pull request template exists at .github/pull_request_template.md
- PropertyCard is in its own file with PropTypes for all props
- Folder structure matches the required layout
- npm run lint passes with no errors
- No console.log, commented code, or unused imports remain

Week 10 Checkpoint

- ☐ Git history tells a clear story of the project development
- ☐ Code is organized and linter passes

- ☐ You can walk through the folder structure and explain each file's responsibility



Week 11: Comprehensive Testing & Documentation

Goal: 70%+ test coverage on critical paths and a complete README.

Requirements

65. Backend: Write tests for all properties route handlers using Jest + Supertest, mocking the database pool
66. Frontend: Write tests for PropertyFilters, Pagination, and PropertyCard components
67. Achieve 70%+ line coverage on the backend routes and frontend components
68. Write a comprehensive README.md covering setup, architecture, API reference, and known issues
69. Add inline comments explaining non-obvious logic (query building, photo parsing, pagination math)

Backend Test Requirements

Your backend tests must cover:

- GET /api/properties — success, pagination, each filter type, invalid inputs
- GET /api/properties/:id — success, 404, invalid ID
- GET /api/properties/:id/openhouses — success, empty results, 404 for unknown property

Frontend Test Requirements

Your frontend tests must cover:

- PropertyFilters: renders all inputs, calls onSearch with correct values, clear resets form
- Pagination: disabled states, page number clicks, ellipsis, hidden when 1 page
- PropertyCard: renders property data, clicking navigates to detail page

README Requirements

Your README must include:

- Project description and screenshot
- Tech stack with versions
- Local setup instructions (step by step from a fresh machine)
- API endpoint reference with example requests and responses
- Database schema summary (table names, key columns, relationships)
- Known issues and future improvements

Acceptance Criteria

- npm test in both backend/ and frontend/ passes with no failures
- Coverage report shows 70%+ for statements on critical files
- README allows a new developer to set up and run the project without asking questions
- API reference documents every endpoint with example request and response
- Comments explain the "why" not just the "what"

Week 11 Checkpoint

- ☐ All tests pass
 - ☐ Coverage targets met
 - ☐ README is complete — have someone else try to set up the project using only the README
-

Week 12: Final Demo & Presentation

Goal: A polished live demo and a technical presentation to the team.

Before your final presentation, you must reach out to your Team Lead to get fresh versions of the `rets_property` and `rets_openhouse` tables and replace your current versions.

Why this matters:

Image URLs (especially for sold properties) expire over time
Using older tables will result in broken images in your dashboards/apps

Action Required:

Message your Team Lead to obtain the latest tables

Replace your existing tables with the new versions

Verify that images are loading correctly before presenting

Final Deliverables

- Working application running locally — both backend and frontend
- Clean Git repository with full commit history
- All tests passing with 70%+ coverage
- Complete README and inline documentation
- 15-minute technical presentation (see structure below)

Presentation Structure (15 minutes)

1. Live Demo (5 minutes)

- Walk through the full user flow: search → filter → paginate → view detail → see map → open photos in lightbox
- Show your advanced feature
- Demonstrate error handling (take the backend down mid-demo)

2. Architecture Deep Dive (5 minutes)

- Draw the data flow from browser to database and back
- Explain one technical decision you made and why
- Show one bug you encountered and how you debugged it

3. Code Walkthrough (5 minutes)

- Walk through your most complex piece of code
- Show your test suite and explain what it covers
- Describe what you would do differently with more time

Evaluation Rubric

Each category is scored 1-5:

- **Functionality (25%)** — All features work correctly, edge cases handled, no crashes
- **Code Quality (20%)** — Clean, readable, well-organized, no dead code
- **Testing (20%)** — Coverage targets met, meaningful tests, edge cases covered
- **Architecture (15%)** — Correct separation of concerns, proper use of REST conventions
- **Problem Solving (10%)** — Can explain debugging process, identified root causes
- **Documentation (10%)** — README is complete and accurate, comments are useful

Interview Prep

During your final demo, be ready to answer these types of questions:

- Walk me through what happens when a user types "Portland" and clicks Search.
 - Why did you use parameterized queries? What would happen without them?
 - What does EXPLAIN tell you about a query? What did you find?
 - How does React state work? Why does changing a filter reset the page number?
 - What would you change about your architecture if you had to support 10,000 concurrent users?
 - Describe a bug you spent the most time on. How did you find it?
-

Appendix A: Known Issues in the Codebase

Part of working on a real project is inheriting someone else's code with bugs. The following issues exist in the provided SQL data and initial setup. You are expected to discover and handle all of them:

Database Issues

- L_Photos values are not always valid JSON — some rows have null, empty string, or malformed arrays
- Some properties have missing or zero lat/lon values — your map component must handle this gracefully
- City names have inconsistent casing (e.g. "portland", "Portland", "PORTLAND") — queries must normalize
- The all_data column in rets_openhouse is a JSON blob — OpenHouseRemarks must be extracted from it, it is not a standalone column

Frontend Issues to Handle

- L_SystemPrice can be null or zero for some properties — your display code must handle this
- L_Keyword2 (beds) and LM_Dec_3 (baths) can be null — do not display "null beds"
- LMD_MP_Latitude and LMD_MP_Longitude can be missing — the map must conditionally render

Sort Feature Bug (if you chose Option 1)

- The sort field whitelist must use actual SQL column names, not RESO names. Using wrong names silently returns unsorted results with no error

Appendix B: Optional Deployment

Deploying is not required but is a strong resume addition. If you finish early, deploy using one of these paths:

Option A: Simple (Recommended)

- Backend → Render (free tier)
- Database → PlanetScale or Railway (MySQL hosting)
- Frontend → Vercel (free tier)

Option B: Full Cloud

- Database → PlanetScale
- Backend → Railway
- Frontend → Netlify

For either option, you will need to update your backend `.env` with cloud database credentials and update the frontend API base URL to point to your deployed backend.